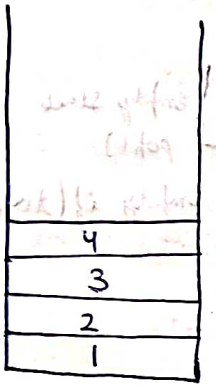
Lecture (64)

Stacks

↳ Data Structure

Plates

LIFO → Last in first out



Stack

insert ⇒ push operation (1, 2, 3, 4)

remove ⇒ pop operation (4, 3, 2, 1)

LIFO order

Peek operation ⇒ gives top element

STL

creation ⇒ Stack<int> s;

push ⇒ s.push(2);

pop ⇒ s.pop();

peek/Top ⇒ s.top(); → returns top element

is empty ⇒ s.empty(); → returns T/F

Size ⇒ s.size();

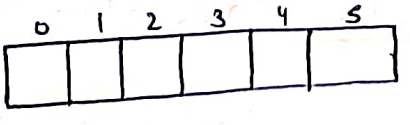
Stack implementation ⇒ class stack create

↳ Code:-

↳ By array

↳ By Linked list (H.W.)

~~class Stack~~



arr

int top → index

& we also need array

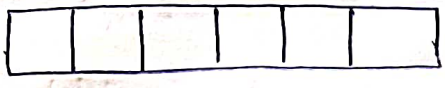
int top

int *arr

int size

Push operation

in starting top = -1



push (22)

↳ first check is space available → T (Insert) → top++; arr[top] = 22;

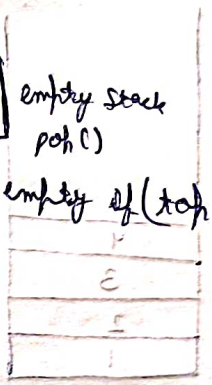
→ F (Stack overflow)

pop()

↳ check element is present → F (Stack underflow) → empty stack pop()

→ T → top--;

Stack empty if (top < 0)

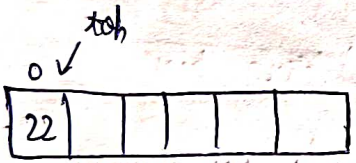


empty()

↳ if top = -1 ⇒ Stack empty

top()

↳

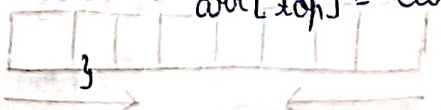


return arr[top];

also check cond^n (top < size)

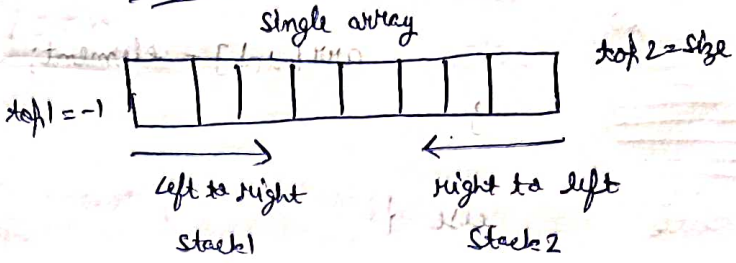
```
class Stack {
    public:
        int * arr;
        int top;
        int size;

        Stack (int size) {
            this -> size = size;
            arr = new int[size];
            top = -1;
        }
```

```
void push (int element) {
    if (size - top > 1) {
        top++;
        arr[top] = element;
    }
    else {
        cout << "Stack overflow" << endl;
    }
}

void pop() {
    if (top >= 0) {
        top--;
    }
    else {
        cout << "Stack Underflow" << endl;
    }
}

int peek() {
    if (top >= 0)
        return arr[top];
    else {
        cout << "Stack is empty" << endl;
        return -1;
    }
}

bool isEmpty() {
    if (top == -1)
        return true;
    else
        return false;
};
```



Qu- Implement 2 stacks in an array.

Approach

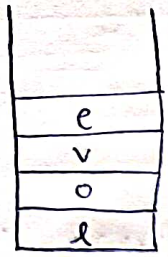



```
Class TwoStack {

    int * arr;
    int top1;
    int top2;
    int size;

public:

    TwoStack(int s) {
        this -> size = s;
        top1 = -1;
        top2 = s;
        arr = new int[s];
    }

    void push1(int num) {
        if (top2 - top1 > 1) {
            top1++;
            arr[top1] = num;
        }
        else {
            cout << " Stack overflow " << endl;
        }
    }

    void push2(int num) {
        if (top2 - top1 > 1) {
            top2--;
            arr[top2] = num;
        }
```

```
        else {
            cout << " stack overflow " << endl;
        }
    }

    int pop1() {
        if (top1 >= 0) {
            int ans = arr[top1];
            top1--;
            return ans;
        }
        else {
            return -1;
        }
    }

    int pop2() {
        if (top2 < size) {
            int ans = arr[top2];
            top2++;
            return ans;
        }
        else {
            return -1;
        }
    }
};
```

Lecture (55)

Qu- Reverse a string using stack



"love"

ans = " "

pop ⇒ ans = evol

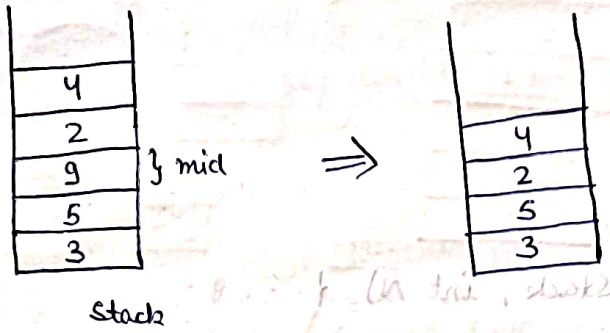
LIFO

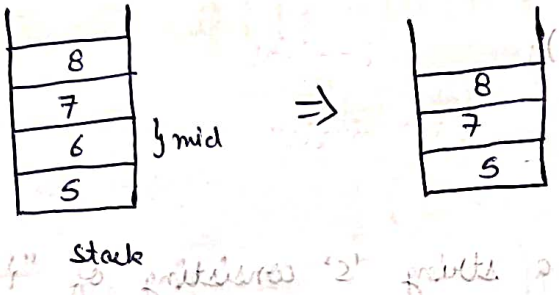
```
int main() {

    string str = "babbar";

    stack<char> s;

    for( int i=0; i< str.length(); i++) {

        char ch = str[i];

        s.push(ch);

    }

    string ans = "";

    while(! s.empty()) {

        char ch = s.top();

        ans.push_back(ch);

        s.pop();

    }

    cout << " answer is : " << ans << endl;

    return 0;

}
```

T.C. = O(N)

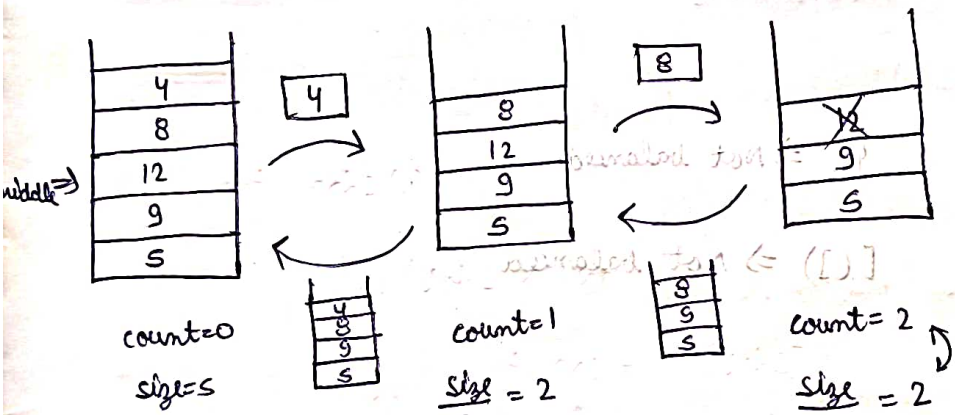
S.C. = O(N)

Qu- Delete middle element from stack





Approach



so, pop()

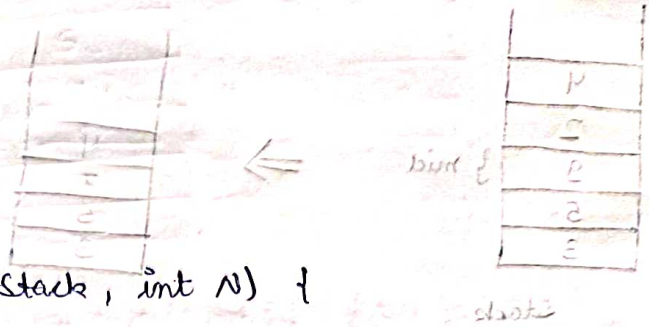
```
void solve (stack<int>& inputStack, int count, int size) {

    // Base case
    if (count == size/2) {
        inputStack.pop();
        return;
    }

    int num = inputStack.top();
    inputStack.pop();
```

// Recursive call

solve (inputStack, count+1, size);

inputStack.push(num);

}

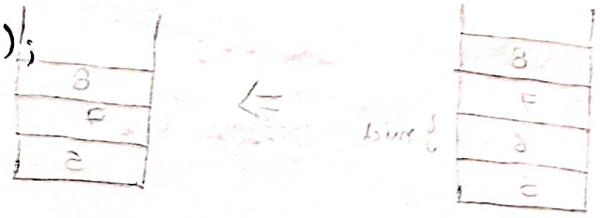


Void deleteMiddle (stack<int> & inputStack, int N) {

int count = 0;

solve(inputStack, count, N);

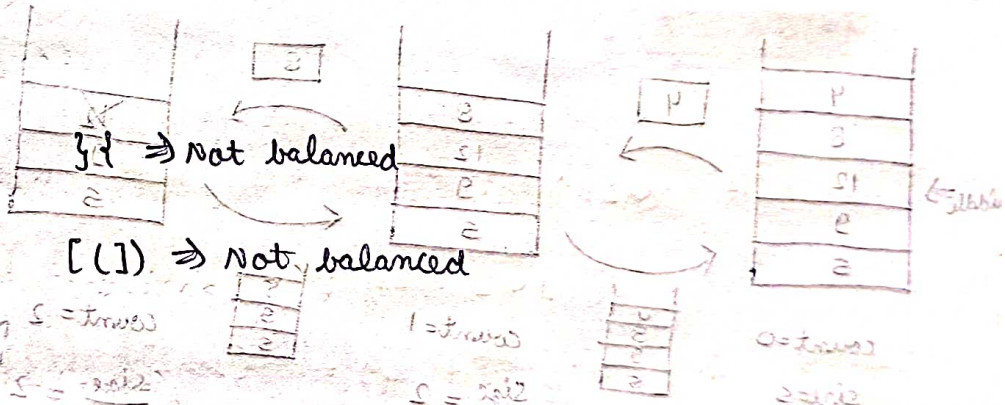
}

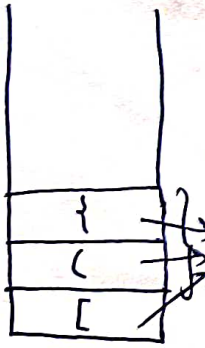


Qu- Valid Parentheses, you are given a string 's' consisting of "{", "}", "(", ")", "[", "]". Return true if the given string s is balanced else return false.

{} ⇒ Balanced

{()} ⇒ Balanced





⟶ if at the end stack empty → True

→ else false

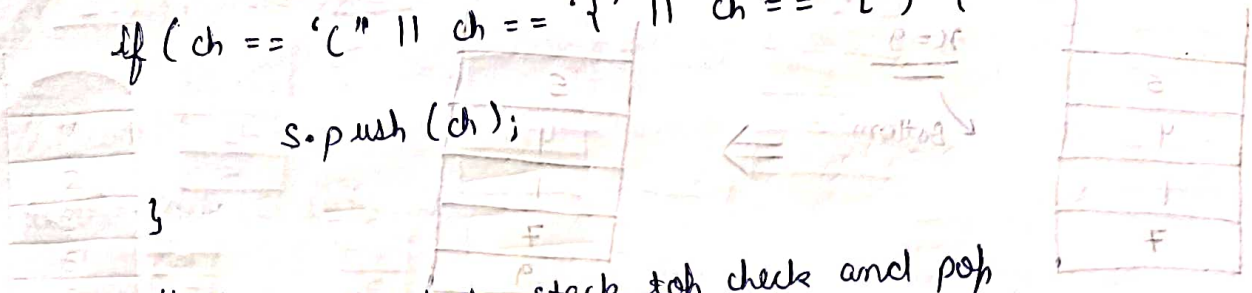
bool isValidParenthesis (string expression) {

stack<char> s;

for (int i=0; i< expression.length(); i++) {

char ch = expression[i];

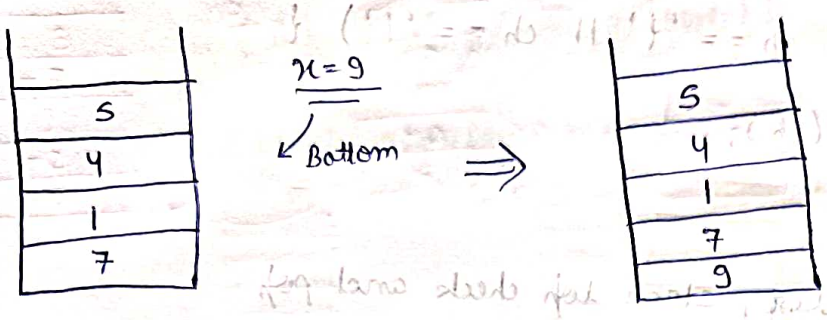
```
// if opening bracken, stack push
if (ch == '(' || ch == '{' || ch == '[') {
        s.push(ch);
}
// if close bracket, stack top check and pop
else {
    if (!s.empty()) {
        char top = s.top();
        if (ch == ')' && top == '(' || ch == '}' && top == '{'
            || ch == ']' && top == '[') {
            s.pop();
        }
        else {
            return false;
        }
    }
    else {
        return false;
    }
}
if (s.empty()) {
    return true;
}
else {
    return false;
}
}
```



T.C. = O(n)

S.C. = O(n)

Qu- Insert an element at it's bottom in a given stack

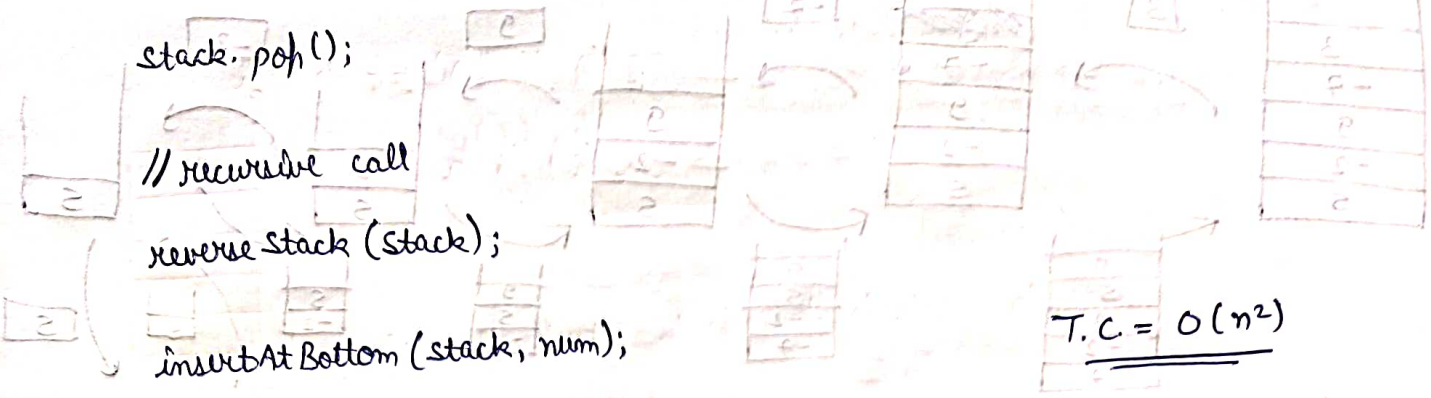



```
Stack<int> pushAtBottom (stack<int>& myStack, int x) {

    // Base Case
    if (myStack.empty()) {
        myStack.push(x);
        return myStack;
    }
    int num = myStack.top();
    myStack.pop();

    // Recursive call
    pushAtBottom (myStack, x);

    myStack.push(num);
    return myStack;
}
```

Qu- Reverse stack using recursion

```
Void reverseStack (stack<int>& stack) {

    // base case
    if (stack.empty()) {
        return;
    }
```

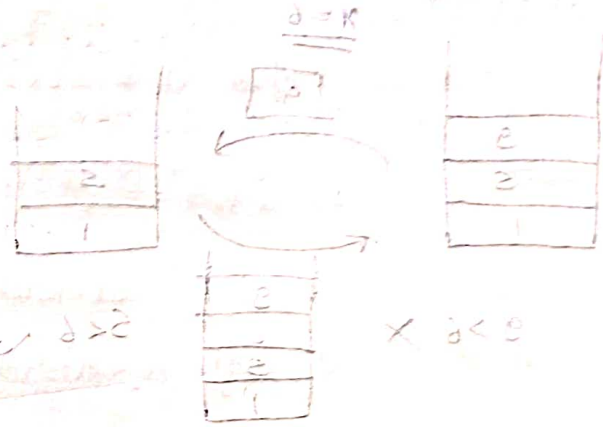
```
    int num = stack.top();
    stack.pop();
    // recursive call
    reverseStack(stack);
    insertAtBottom(stack, num);
}
```



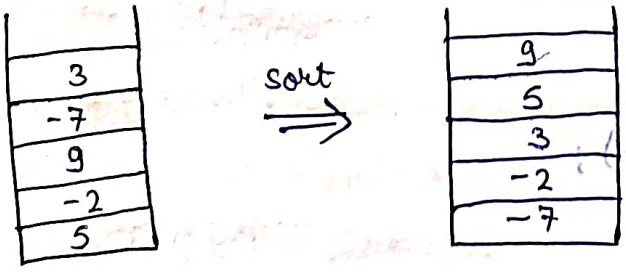
T.C = $O(n^2)$

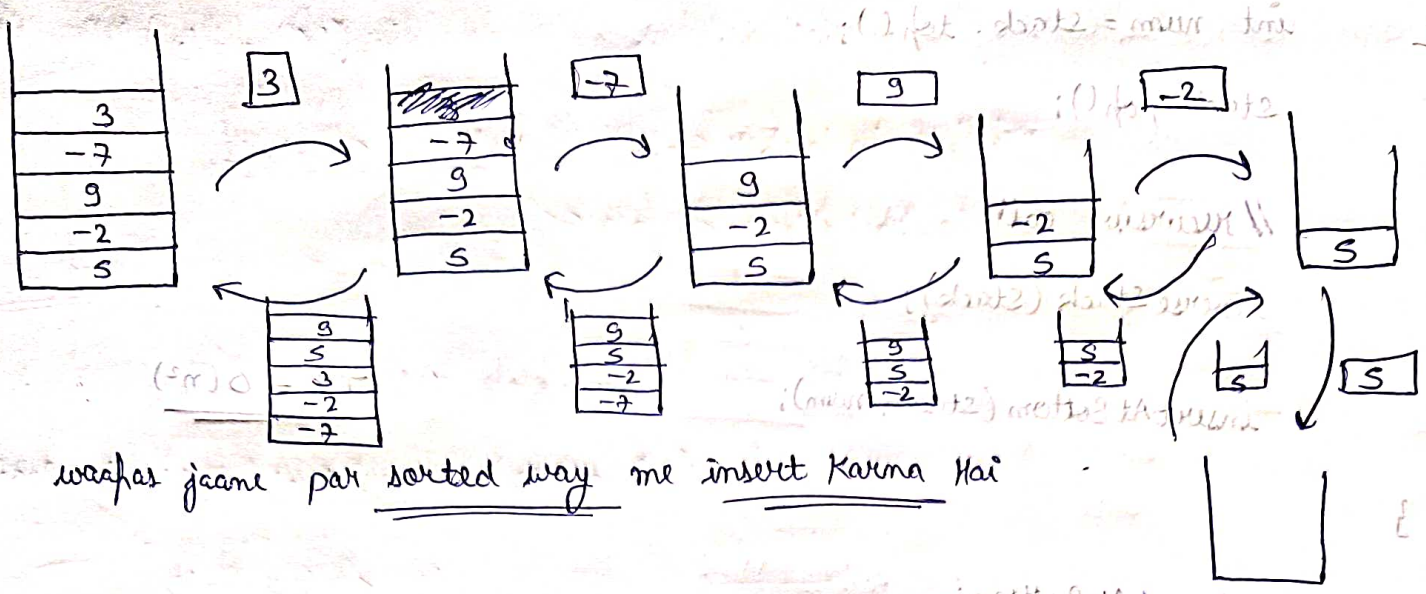
Code for insertAtBottom :-

```
void insertAtBottom(stack<int> &s, int element) {
    // basecase
    if(s.empty()) {
        s.push(element);
        return;
    }
    int num = s.top();
    s.pop();
    // recursive call
    insertAtBottom(s, element);
    s.push(num);
}
```



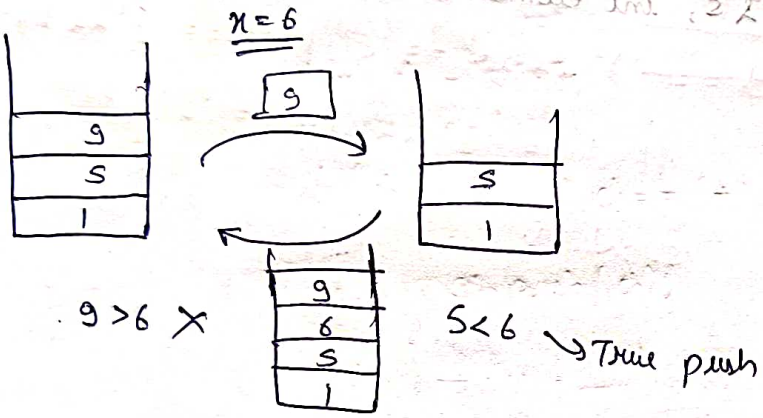
Qu- Sort stack in descending order using recursion



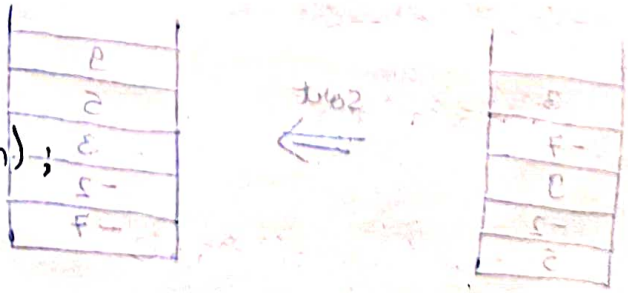


wapas jaane par sorted way me insert karna hai

Sorted insert




```
Void sortedInsert (stack<int> &stack, int num) {

    //base case

    if (stack.empty() || (!stack.empty() && stack.top() < num)) {

        stack.push(num);

        return;

    }

    int n = stack.top();

    stack.pop();

    // recursive call

    sortedInsert (stack, num);

    stack.push(n);
```



```
void sortStack (stack<int> &stack) {

    // base case
    if (stack.empty()) {
        return;
    }

    int num = stack.top();
    stack.pop();

    // recursive call
    sortStack (stack);

    sortedInsert (stack, num);
}
```

Qu- Redundand Brackets, (a pair of brackets is said to be redundant when a subexpression is surrounded by needless/useless brackets). Return true if the given expression contains a pair of redundant brackets, else return false.

I/p ⇒ ((a+b))

O/p ⇒ Yes

I/p ⇒ (a+(b*c))

O/p ⇒ No

Approach i/p → string = "(a+(b*c))"

```
for (' ' == ch) || '\' == ch) {
    ch = str[i];
    if (ch → '(' || operator)
        stack.push(ch)
    else if (')' || lower letter)
```

if (ch == ')') {

Stack → operator → Yes → redundant nahi hai

→ No → redundant hai

}

}

```
bool findRedundantBrackets(string &s) {

    stack<char> st;

    for (int i=0; i<s.length(); i++) {

        char ch = s[i];

        if (ch == '(' || ch == '+' || ch == '-' || ch == '*' || ch == '/') {

            st.push(ch);

        }

        else {

            // ch mein to ')' or lowercase letter

            if (ch == ')') {

                bool isRedundant = true;

                while (st.top() != '(') {

                    char top = st.top();

                    if (top == '+' || top == '-' ||

                        top == '*' || top == '/') {

                        isRedundant = false;

                    }

                    st.pop(); // '('

                }
```

}

```
            if (isRedundant == true) {
                return true;
            }
            st.pop();
        }
    }
}
return false;
}
```

T.C. $\Rightarrow O(n)$

Qu- Minimum cost to make string valid

~~Ex~~ ⇒ {{}{} — Valid ; {{}} — Not Valid ; {}{ — Not Valid ; {{}} — Valid

For Valid

Condⁿ → no of open brace = no of close brace
→ every close brace should have open brace before it

str = {{{} —(kitne brackets ko minimum reverse karu ki expression valid ho jaaye)→ {}{{} (i.e. reversing at index 1)

Logic :- if (str.length() == odd) → return -1

Invalid ⇒ ① {{{{---
② }}}}---
③ ---}}}{{{---

Algo :- ① odd → -1
② i/p string → valid part remove
↳ remaining part → above 3 types

⑧ }}}{{{

a → count of close braces = 3

b → count of open braces = 3

Case ① :- }{}{ → b=4 (even)

{}{} → ans = 2 ($\frac{b}{2}$)

}{{{{{ (odd) ↳ return -1

Case ② :- }}}}}} → a=4 (even)

{}{}{} → ans = 2 ($\frac{a}{2}$)

}}}}}} (odd) ↳ return -1

Case ③ :- }}{{}{ → a=3, b=3 (odd odd)

{}{}{}{} → ans = 4

$\left(\frac{a+1}{2}\right) + \left(\frac{b+1}{2}\right)$

}}}{{{ → a=4, b=4 (even even)

{}{}{}{} → ans = 4

$\left(\frac{a+1}{2}\right) + \left(\frac{b+1}{2}\right)$

So, final answer in all cases = $\left(\frac{a+1}{2}\right) + \left(\frac{b+1}{2}\right)$

```
int findMinimumCost (string str) {

    //odd condition
    if (str.length() % 2 == 1) {
        return -1;
    }

    stack <char> s;

    for(int i=0; i < str.length(); i++) {
        char ch = str[i];
        if (ch == '{') {
            s.push(ch);
        }
```

```
else {
    if (!s.empty() && s.top() == '{') {
        s.pop();
    }
    else {
        s.push(ch);
    }
}
int a = 0, b = 0;
while (!s.empty()) {
    if (s.top() == '}') {
        b++;
    }
    else {
        a++;
    }
    s.pop();
}
int ans = (a+1)/2 + (b+1)/2;
return ans;
}
```

Lecture (58)

Qu- Next smaller element i.e. Har element ke liye first smaller element pakda do

i/p ⇒ [2, 1, 4, 3]

o/p ⇒ {1, -1, 3, -1}

Approach ①

[2, 1, 4, 3]

2 → {1,4,3} ⟹ (n-1) comparisons

1 → {4,3} ⇒ (n-2) comparisons

4 → {3} ⇒ 1 comparison

3 → {} ⇒ 0 comparisons

```
for (int i=0; i<n; i++) {
    for (int j=i+1; j<n; j++) {
    }
}
```

T.C ⇒ $O(n^2)$

Brute force approach

Approach ②

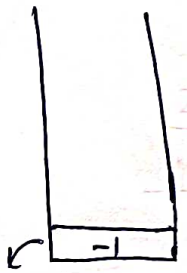
comparison from right side ←

[2, 1, 4, 3]

Using Stack

curr

3 → next smaller element



Starting me (-1) store karlo

Stack

s.top()

T.C. ⇒ $O(n)$

chotta → ans store

→ s.push(curr)

Bada → chotta dhund ke lao

while (element (s.top()) >= curr) {

s.pop()

}

→ s.top() → chotta

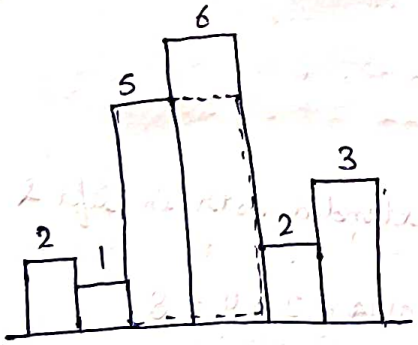
→ ans = s.top()

→ s.push(curr)

```
vector<int> nextSmallerElement(vector<int> &arr, int n) {

    stack<int> s;

    s.push(-1);

    vector<int> ans(n);

    for(int i=n-1; i>=0; i--) {

        int curr = arr[i];

        while(s.top() >= curr) {

            s.pop();

        }

        ans[i] = s.top();

        s.push(curr);

    }

    return ans;

}
```

Qu- Largest Rectangular Area in Histogram (width = 1)

i/p ⇒



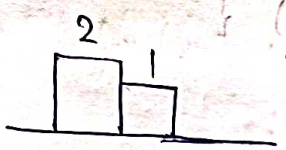
o/p ⇒ 10

Brute force Approach

area = length * breadth ⇒ max.

extend the bar by taking each one by one & return the max. area

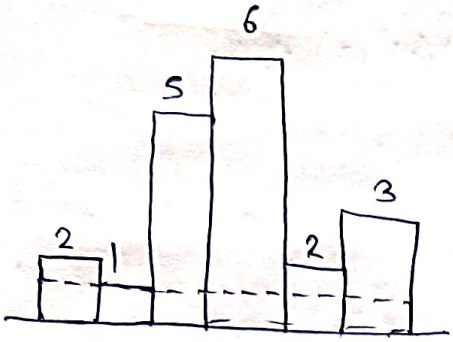
① 2



Not extended (neither left nor right)

So, area = 2 × 1 = 2

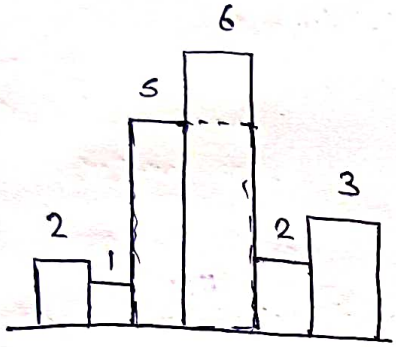
② 1



1 is extended both in left & right

So, area = 1 × 6 = 6

③ 5



5 is extended in right

So, area = 5 × 2 = 10

④ 6

6 is not extended (neither left nor right)

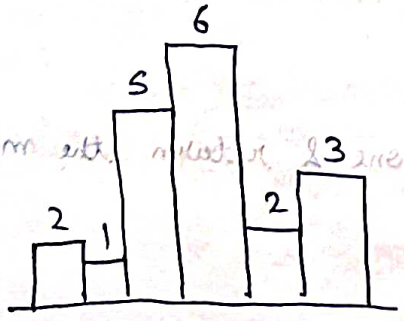
So, area = 6 × 1 = 6

⑤ 2

2 is extended both in left & right

So, area = 2 × 4 = 8

⑥ 3



3 is not extended (neither left nor right)

So, area = 3 × 1 = 3

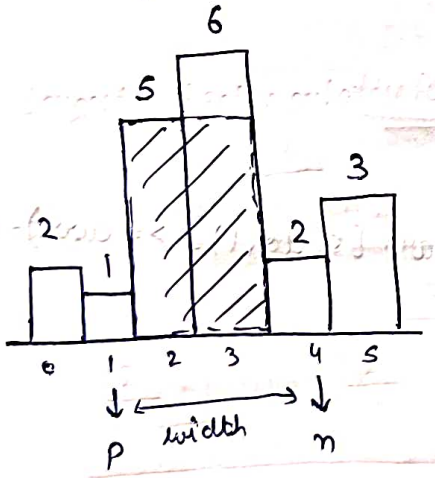
Code for this approach

```
for (int i=0; i<n; i++) {

    while (left) {
    ==========
    }

    while (right) {
    ==========
    }

    new area;

    area = max (area, new-area);

}
```

T.C. = $O(n^2)$

Approach ②

By using stack



width = n − p − 1

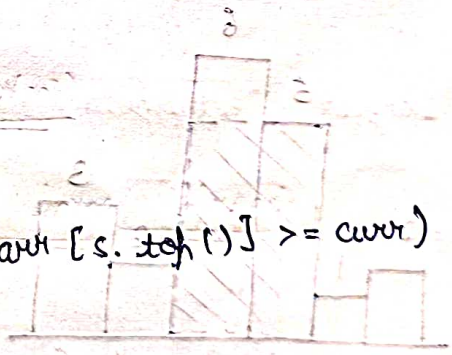
= 6 − 4 − 1

= 1

area = length * width

```
vector <int> nextSmallerElement (vector<int> arr, int n) {

    stack<int> s;

    s.push(-1);

    vector<int> ans(n);
```

```
for (int i=n-1; i>=0; i--) {

    int curr = arr[i];

    while (s.top() != -1 && arr[s.top()] >= curr) {

        s.pop();

    }

    ans[i] = s.top();

    s.push(i);

}

return ans;

}

vector<int> prevSmallerElement (vector<int> arr, int n) {

    stack<int> s;

    s.push(-1);

    vector<int> ans(n);

    for (int i=0; i<n; i++) {

        int curr = arr[i];

        while (s.top() != -1 && arr[s.top()] >= curr) {

            s.pop();

        }

        ans[i] = s.top();

        s.push(i);

    }

    return ans;

}
```



```
int largestRectangleArea (vector<int>& heights) {
    int n = heights.size();
    vector<int> next(n);
    next = nextSmallerElement(heights, n);

    vector<int> prev(n);
    prev = prevSmallerElement(heights, n);

    int area = INT_MIN;

    for(int i=0; i<n; i++) {

        int l = heights[i];

        if (next[i] == -1) {
            next[i] == n;
        }

        int b = next[i] - prev[i] -1;

        int newArea = l* b;

        area = max(area, newArea);

    }

    return area;

}
```

T.C. = O(n)

S.C. = O(n)

Lecture (57)

Qu- Celebrity Problem, A celebrity is a person who is known to all but does not know anyone at a party. If you go to a party of N people, find if there is a celebrity in the party or not.

A square N×N matrix M[][] is used to represent people at the party such that if an element of row i and column j is set to 1 it means i^{th} person knows j^{th} person. Here M[i][i] will always be 0.

i/p ⇒

	0	1	2
0	0	1	0
1	0	0	0
2	0	1	0

① Ignore Diagonal

1 → knows no one

everyone knows 1

So, 1 is a celebrity

Approach ①

① Celebrity row → all 0

② Celebrity column → all 1's except diagonal element

i.e. I check n element ki rows & colm

```
for ( int i=0; i<n; i++) {
    row  } loop → O(n)
    colm
                + → O(n)
}
```

T.C. = $O(n^2)$

Brute force Approach

```
if (bool__) {
    1 or not
}
```

Approach ②

Algo:- ① put all element inside stack

② jab tak stack size != 1 tab tak 2 element nikaal lo

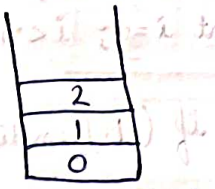
↳ A → s.top() → s.pop()

↳ B → s.top() → s.pop()

③ if A knows B

A → not celebrity

B → wapas push karo



Stack

④ if B knows A

B → not celebrity

A → wapas push karo

⑤ Jo single element Bacha hua hai vo ek potential celebrity ho sakta hai

⑥ Verify

↳ celebrity knows no one ⇒ row check → all 0's

↳ everyone knows celeb ⇒ col^m ⇒ all 1 except diagonal element

Code:-

```
int celebrity (vector<vector<int>> & M, int n) {

    Stack < int > s;
    for(int i=0; i<n; i++) {
        s.push(i);
    }

    while (s.size() >1) {
        int a = s.top();
        s.pop();
        int b = s.top();
        s.pop();
        if (knows(M,a,b)) {
            s.push(b);
        }
        else {
            s.push(a);
        }
    }
```

T.C. = O(n)

```
int candidate = s.top();
bool rowCheck = false;
int zeroCount=0;
for (int i=0; i<n; i++) {
    if (M[candidate][i] ==0) {
        zeroCount ++;
    }
}
if (zeroCount ==n) {
    rowCheck = true;
}
bool colCheck = false;
int oneCount =0;
for (int i=0; i<n; i++) {
    if (M[i][candidate] ==1) {
        oneCount++;
    }
}
if (oneCount == n-1) {
    colCheck = true;
}
if (rowCheck == true && colCheck == true) {
    return candidate;
}
else {
    return -1;
}
}
```

Knows fxn :-

```
bool knows(vector<vector<int>> &M, int a, int b, int n) {
    if (M[a][b] == 1) {
        return true;
    }
    else {
        return false;
    }
}
```

H.W.

More approaches to celebrity problem

Qu- Given a binary matrix M of size n x m. Find the maximum area of a rectangle formed only of 1s in the given matrix.

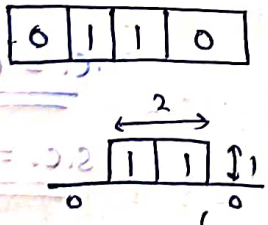
i/p ⇒

0	1	1	0
1	1	1	1
1	1	1	1
1	1	0	0

o/p ⇒ 8

Approach

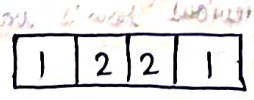
Algo:- ① 1st row

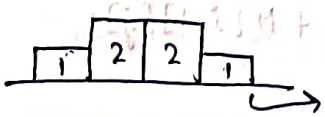


↳ max. area = 2

(Largest area in histogram wala logic lagaya)

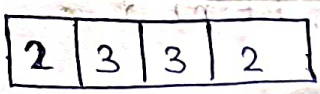
② 1st & 2nd row

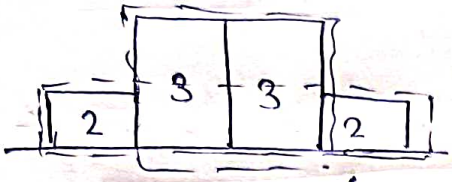




} Upar waali row ke element ko add karo current row ke saath agar base 0 nhi hai to

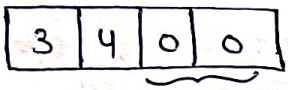
③ 1st - 3 rows



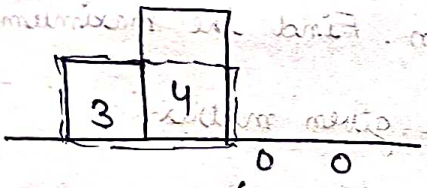


↳ max. area = max (6, 8) = 8

④ now all rows (Taking last row as base)

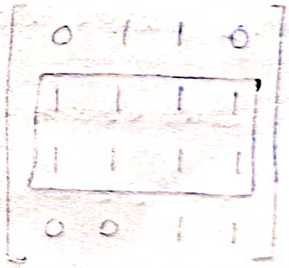


Because base hi 0 hai
To building hawa me
Thodi banegi



↳ max. area = 6

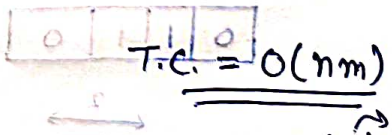
So, max. area out of all rows = 8



Code:

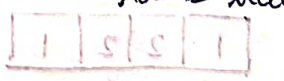
Copy the full previous code (Histogram wala)

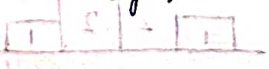
```
int maxArea(int M[MAX][MAX], int n, int m) {
    // Compute area for first row
    int area = largestRectangleArea(M[0], m);
    for(int i=1; i<n; i++) {
        for(int j=0; j<m; j++) {
            // row update: by adding previous row's value
            if(M[i][j] != 0) {
                M[i][j] = M[i][j] + M[i-1][j];
            }
```



T.C. = O(nm)

S.C. = O(m)



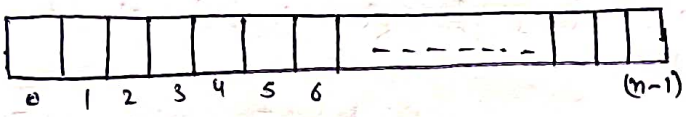


```
        else {
            M[i][j] = 0;
        }
    }
    // entire row is updated now
    area = max(area, largestRectangleArea(M[i], m));
}
return area;
}
```

Lecture (58)

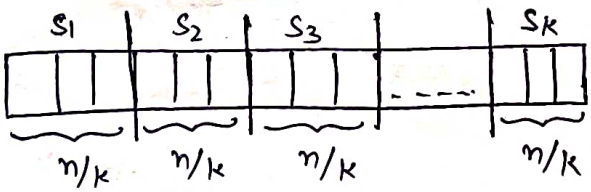
Qu- Implement N stacks in an array

Approach (1)



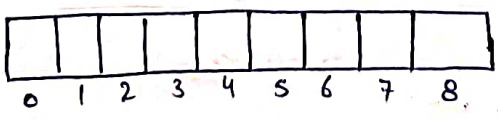
I have to implement K stacks, so divide array into K parts

$$1 \text{ part size} = \frac{n}{K}$$



Drawback:- Not Space Optimised

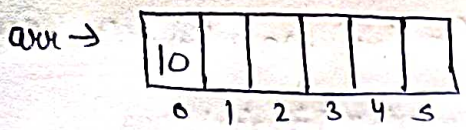
Approach (2)



9 sized array
3 stacks implement

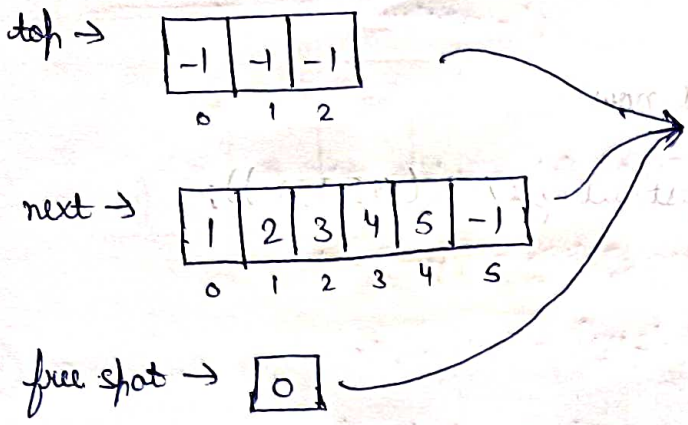
top[] → represent index of top element

next[] → point to next element after stack top
↳ point to next free space / block



size of array = 6

no of stacks = 3



⇒ push (10, 1)
x m

① // find index

int index = freespot;

② // freespot update

freespot = next[index];

③ // insert in array

arr[index] = x;

④ // update next (purana top)

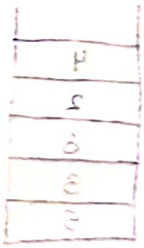
next[index] = top[m-1];

⑤ // update top

top[m-1] = index;

```
Class NStack {
    int * arr;
    int * top;
    int * next;
    int n, s;
    int freespot;
```

```
public:
    NStack (int N, int S) {
        n = N;
        s = S;
        arr = new int [s];
        top = new int [n];
        next = new int [s];
        for (int i=0; i<n; i++) {
            top [i] = -1;
        }
        for (int i=0; i<s; i++) {
            next [i] = i+1;
        }
        next [s-1] = -1;
        freespot = 0;
    }

    bool push (int x, int m) {
        if (freespot == -1) {
            return false;
        }
        int index = freespot;
        freespot = next [index];
        arr [index] = x;
        next [index] = top [m-1];
        top [m-1] = index;
        return true;
    }
```



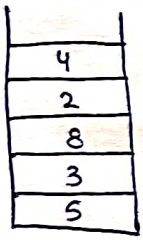
```
int pop(int m) {
    if (top[m-1] == -1) {
        return -1;
    }
    int index = top[m-1];
    top[m-1] = next[index];
    next[index] = freeslot;
    freeslot = index;
    return arr[index];
}
};
```

T.C. = O(1)

S.C. = O(S+n)

Lecture (59)

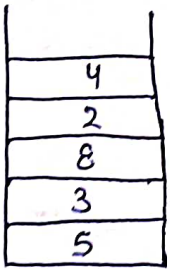
Qu- Design a stack that supports getMin() in O(1) time and O(1) extra space.



i/p stack

getMin() → 2
pop()
getMin() → 2
pop()
getMin() → 3

Approach



curr
5, 3, 8, 2, 4
push ①

```
int mini = INT_MAX;
mini = min(mini, curr) → 5
     = min(5,3) → 3
     = min(3,8) → 3
     = min(3,2) → 2
     = min(2,4) → 2
```

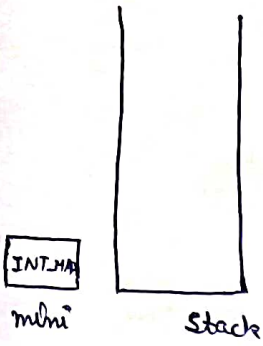
But here I have to store min. values at each step so space used ⇒ O(n)

T.C. ⇒ O(1)

S.C ⇒ O(1) i.e. solve using variable

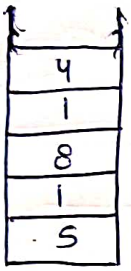
Approach ②

5, 3, 8, 2, 4



push operation
- ↳ overflow cond"
- ↳ for 1st element → normal push & mini update
- ↳ baaki element
  - ↳ if (curr < mini)
    - ↳ val = 2 * curr − mini
    - ↳ push val
    - ↳ mini update
  - ↳ else
    - ↳ normal push

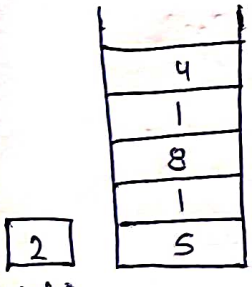
mini = min (mini, curr)
= min (INT_MAX, 5)
= 5



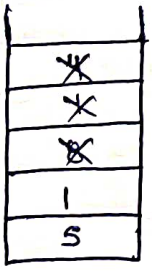
2 * 3 − 5 = 1

2 * 2 − 3 = 1

pop operation
- ↳ underflow cond"
- ↳ if (curr > mini)
  - ↳ normal pop()
- ↳ else
  - ↳ val = 2 * mini − curr



mini = val;
pop();



val = 2 * 2 − 1 = 3
mini = 3

val = 2 * 3 − 1 = 5
mini = 5

getMin()
- ↳ mini stored return

push → 2 * curr − mini

pop → 2 * mini − curr

Taaki mai curr min. ka use karke prev. min. ko find kar saku

```
class SpecialStack {

    Stack<int> s;
    int mini;

public:

    void push(int data) {
        if (s.empty()) {
            s.push(data);
            mini = data;
        }
        else {
            if (data < mini) {
                int val = 2 * data - mini;
                s.push(val);
                mini = data;
            }
            else {
                s.push(data);
            }
        }
    }

    int pop() {
        if (s.empty()) {
            return -1;
        }
        int curr = s.top();
        s.pop();
        if (curr > mini) {
            return curr;
        }
```

```
        else {
            int prevMin = mini;
            int val = 2* mini - curr;
            mini = val;
            return prevMin;
        }
    }
    int top() {
        if (s.empty()) {
            return -1;
        }
        int curr = s.top();
        if (curr < mini) {
            return mini;
        }
        else {
            return curr;
        }
    }
    bool isEmpty() {
        return s.empty();
    }
    int getMin() {
        if (s.empty()) {
            return -1;
        }
        return mini;
    }
};
```